StoreManager

Documento de estudio y sustentación
Entrega 2 — HTML, CSS y JavaScript (sin frameworks)

Proyecto universitario
Contenido: estructura del proyecto, cumplimiento de requerimientos,
cómo se conectan los HTML, CSS y JS, y los flujos clave.

1. ¿Qué es el proyecto?

StoreManager es una pequeña aplicación web para gestionar el inventario de una tienda. Tiene cuatro páginas: una de inicio con un resumen del día, el listado de productos con búsqueda, filtro por categoría y paginación, y dos formularios (agregar y editar producto) con validación y carga de imagen.

El desarrollo es con HTML, CSS y JavaScript puros, sin frameworks ni librerías de UI. Solo se usan dos recursos externos: la fuente Inter (Google Fonts) y los íconos Lucide (CDN).

2. Estructura del proyecto

```
MAQUETA PROYECTO-WEB/
|
|-- index.html             Pagina de inicio (resumen)
|-- inventario.html        Listado de productos
|-- agregar-producto.html  Formulario de nuevo producto
|-- editar-producto.html   Formulario de edicion (recibe ?id=)
|
|-- css/
|   |-- variables.css      Colores, sombras, fuente (:root)
|   |-- base.css           Reset y estilos generales
|   |-- layout.css         Estructura: sidebar, topbar, grillas
|   |-- components.css     Tarjetas, tabla, botones, formulario, modal
|
|-- js/
|   |-- utilidades.js      Helpers: $, moneda, esc, ...
|   |-- datos.js           Constantes y productos de ejemplo
|   |-- repositorio.js     UNICO acceso a los datos (en memoria)
|   |-- componentes.js     Menu movil, imagen, confirmar, validacion
|   |-- inicio.js          Logica de la pagina Inicio
|   |-- inventario.js      Logica de la pagina Inventario
|   |-- formulario.js      Logica de Agregar / Editar
|
|-- img/fondo-lineas.svg   Fondo decorativo del contenido
```

Regla general: un archivo = una responsabilidad. Los archivos JS se cargan en un orden fijo (ver sección 4) porque los scripts clásicos comparten el ámbito global: cada archivo puede usar las funciones de los anteriores.

3. Cumplimiento de los requerimientos de la entrega

Requerimiento de la entrega	¿Cumple?	Dónde se demuestra
Páginas construidas con HTML y CSS, fieles al diseño	SI	4 páginas (index, inventario, agregar-producto, editar-producto) + 4 archivos CSS
Diseño responsive: escritorio, tableta y celular	SI	Media queries a 1000px, 800px y 520px en layout.css y components.css
Interactividad con eventos addEventListener	SI	15 manejadores: menú móvil, buscador, filtro de categoría, paginación, confirmación antes de eliminar, envío del formulario, selección de imagen
Validación de formularios con mensajes en la página (sin alert)	SI	validarProducto() en componentes.js + mostrarError() en formulario.js; el error aparece en rojo bajo cada campo y se quita al escribir

En esta entrega los datos aún no se guardan	SI	repositorio.js mantiene los productos en memoria (productosEnMemoria); al recargar vuelven los datos de ejemplo de datos.js

No cumple / puntos a considerar

• Se carga Lucide (librería de íconos por CDN) y la fuente Inter (Goog
Bootstrap o Tailwind, pero técnicamente son recursos externos. Si el d
íconos se pueden reemplazar por SVG escritos a mano.

• En el menú lateral, los ítems Tienda, Carrito, Ventas y Configu
corresponden a páginas de entregas futuras. Inicio e Inventario sí funcionar

• Los datos de ejemplo viven en js/datos.js y se pintan con JavaScript
especificación lo permite ("pueden mostrar datos de ejemplo") y es necesa

4. ¿Cómo se conectan los HTML, los CSS y los JS?

4.1 HTML -> CSS (en el <head>, en este orden)

```
<link rel="stylesheet" href="css/variables.css">    <!-- 1. variables :root -->
<link rel="stylesheet" href="css/base.css">         <!-- 2. reset y generales -->
<link rel="stylesheet" href="css/layout.css">       <!-- 3. estructura -->
<link rel="stylesheet" href="css/components.css">   <!-- 4. componentes -->
```

El orden importa: primero las variables (las demás hojas las usan con var(--verde), etc.), luego los estilos generales y al final los componentes, que pueden afinar o pisar estilos anteriores. Las cuatro páginas cargan exactamente los mismos cuatro archivos CSS.

4.2 HTML -> JS (al final del <body>, en este orden)

```
<script src="js/utilidades.js"></script>     <!-- helpers puros -->
<script src="js/datos.js"></script>          <!-- constantes y ejemplos -->
<script src="js/repositorio.js"></script>    <!-- acceso a datos -->
<script src="js/componentes.js"></script>    <!-- piezas de interfaz -->
<script src="js/inicio.js"></script>         <!-- o inventario.js / formulario.js -->
```

El orden importa porque son scripts clásicos: todo queda en el ámbito global y cada archivo usa funciones de los anteriores. Por ejemplo, componentes.js usa $ (de utilidades.js) y existeCodigo (de repositorio.js); si se invierte el orden, la página se rompe.

4.3 Qué carga cada página

Página	Archivos JS que carga
index.html	utilidades, datos, repositorio, componentes, inicio
inventario.html	utilidades, datos, repositorio, componentes, inventario
agregar-producto.html	utilidades, datos, repositorio, componentes, formulario
editar-producto.html	utilidades, datos, repositorio, componentes, formulario

Así cada página carga solo lo que necesita: la base común más su propio script.

4.4 Qué lee cada JS del HTML (ids y atributos)

Archivo JS	Elementos del HTML que usa
inicio.js	#saludo, #subtitulo, #fecha, #indicadores, #recientes, #stockBajo
inventario.js	#buscador, #categoria, #filas, #paginas, #resumen y el modal (#modalEliminar, #cancelarEliminar, #confirmarEliminar)
formulario.js	#formulario y sus campos (#codigo, #nombre, #categoria, #precio, #cantidad, #descripcion), #btnImagen, #archivo, #vistaPrevia, #subtitulo, #errorGeneral; lee data-modo="editar" del <body> y el ?id= de la URL
componentes.js	#sidebar, #menuBtn (menú móvil) y el modal de confirmación

Si en el HTML se cambia el id de un elemento, hay que cambiarlo también en el JS (y viceversa). Los r
clase (class) los usa el CSS; los ids los usa el JS.

4.5 Dependencias entre archivos JS

```
utilidades.js    (no depende de nadie: $, moneda, esc, ...)
       ^
       |
datos.js         (constantes: UMBRAL_STOCK_BAJO, CATEGORIAS, PRODUCTOS_BASE)
       ^
       |
repositorio.js   (usa datos.js: cargarProductos, agregarProducto, ...)
       ^
       |
componentes.js   (usa utilidades.js y repositorio.js: menu, imagen,
                    confirmar, validarProducto)
       ^
       |
inicio.js / inventario.js / formulario.js   (usan todos los anteriores)
```

5. Flujos clave (para sustentar)

5.1 Página de inicio (index.html + inicio.js)

- iniciarMenuMovil() activa el menú lateral en celular (se abre con el botón y se cierra al tocar fuera).
- renderEncabezado(): saludo según la hora (Buenos días / Buenas tardes / Buenas noches) y fecha en español (es-CO).
- renderIndicadores(): dibuja las 4 tarjetas de indicadores desde un array de configuración.
- Productos recientes: por ahora con ids fijos [1, 3, 2, 4].
- Productos con stock bajo: se CALCULA comparando cada stock con UMBRAL_STOCK_BAJO (10). No usa ids fijos: si los datos cambian, la vista sigue siendo correcta.

5.2 Inventario (inventario.html + inventario.js)

- Estado: productos (desde cargarProductos()), pagina y filtro = { texto, categoria }.

• render(): filtra por texto (nombre o código) y categoría, calcula las páginas (6 productos por página), dibuja las filas y la paginación.

• El buscador (evento input) y la categoría (evento change) reinician pagina a 1 y vuelven a renderizar.

• Eliminar: al dar clic en el ícono de la papelera se abre un <dialog> de confirmación (mostrarConfirmar). Solo si se confirma, eliminarProducto() lo quita y se re-renderiza la tabla.

5.3 Agregar / Editar (agregar-producto.html, editar-producto.html + formulario.js)

• El <body> trae data-modo="editar" en la página de editar; con eso el mismo formulario sirve para ambos casos (un solo archivo para dos páginas).

• Editar: lee el ?id= de la URL y carga ese producto. Si se abre la página a mano (sin id), carga el primer producto para que la página nunca quede vacía.

• Imagen: el botón abre un input de archivo oculto; se valida el tipo (PNG/JPG) y el tamaño (máximo 5 MB); luego se reduce a 480 px para que quepa en la memoria y se muestra en la vista previa.

• Validación al enviar: código obligatorio y sin duplicados, nombre obligatorio, categoría obligatoria, precio mayor que 0 y cantidad entera no negativa. Los errores se pintan en rojo bajo cada campo (clase .con-error) y se quitan apenas se vuelve a escribir.

• Guardar: agregarProducto() o actualizarProducto() según el modo, y redirige al inventario. En esta entrega los cambios viven solo en memoria (ver sección 3).

6. Buenas prácticas aplicadas (útiles para la sustentación)

• Separación por responsabilidades: utilidades, datos, acceso a datos, componentes y una página por archivo. Cada archivo se explica solo.

• El repositorio es la ÚNICA puerta a los datos: si en la siguiente entrega piden guardar en localStorage o en un servidor, solo cambia repositorio.js; las páginas no se tocan.

• La validación es una función pura: validarProducto() recibe datos y devuelve errores; no sabe nada del DOM ni del almacenamiento.

• Sin alert(): todos los mensajes se pintan en la página (errores bajo cada campo, modal de confirmación, aviso general).

• Se usa esc() para escapar textos antes de insertarlos en el HTML, evitando problemas si un producto tiene caracteres como < o &.

• Código comentado en español y nombres de variables descriptivos (filaProducto, renderPaginacion).

7. Pendientes / siguiente entrega

• Persistencia de datos (localStorage o conexión a un backend). La estructura ya está lista: basta con guardar productosEnMemoria en localStorage dentro de repositorio.js.

• Páginas faltantes del menú: Tienda, Carrito, Ventas y Configuración.

• Si se exige 100% sin librerías: reemplazar los íconos de Lucide por SVG escritos a mano.